

Assignment Kit for Coding Standard



Personal Software Process for Engineers: Part I

The Software Engineering Institute (SEI)
is a federally funded research and development center
sponsored by the U.S. Department of Defense and
operated by Carnegie Mellon University.

This material is approved for public release.
Distribution limited by the Software Engineering Institute to attendees.

Personal Software Process for Engineers: Part I

Assignment Kit for the Coding Standard

Overview

Overview This assignment kit covers the following topics.

Section	See Page
Prerequisites	2
Objectives	2
Coding standard requirements	3
Example coding standard	4
Evaluation criteria and suggestions	7
Coding standard template	8

Prerequisites

Prerequisites

- Read Chapter 4
- Complete Size Counting Standard

Objectives

The objectives of the coding standard are to

- establish a consistent set of coding practices
- provide criteria for judging the quality of the code that you produce
- facilitate size counting by ensuring your programs are written so they can be readily counted
- for LOC counting, require that there be a separate physical line for each logical line of code

Coding standard requirements

Coding standard requirements

Produce, document, and submit a completed coding standard that calls for quality coding practices.

For LOC counting, ensure that a separate physical source line is used for each logical line of code.

Submit the coding standard with your program 2 assignment package.

Example coding Standard

Coding standard example

Pages 5 and 6 of this workbook contain an example C++ coding standard.

Notes about the example

- Since it is an example, tailor it to meet your personal needs.
 - If you have an existing organizational standard, consider using it for the PSP exercises.
-

Continued on next page

Example C++ Coding Standard

Purpose	To guide implementation of C++ programs
Program Headers	Begin all programs with a descriptive header.
Header Format	<pre> /***** /* Program Assignment: the program number */ /* Name: your name */ /* Date: the date you started developing the program */ /* Description: a short description of the program and what it does */ *****/ </pre>
Listing Contents	Provide a summary of the listing contents
Contents Example	<pre> /***** /* Listing Contents: /* Reuse instructions /* Modification instructions /* Compilation instructions /* Includes /* Class declarations: /* CData /* ASet /* Source code in c:/classes/CData.cpp: /* CData /* CData() /* Empty() *****/ </pre>

(continued)

Example C++ Coding Standard (continued)

Reuse Instructions	- Describe how the program is used: declaration format, parameter values, types, and formats. - Provide warnings of illegal values, overflow conditions, or other conditions that could potentially result in improper operation.
Reuse Instruction Example	<pre> /***** /* Reuse instructions */ /* int PrintLine(char *line_of_character) */ /* Purpose: to print string, 'line_of_character', on one print line */ /* Limitations: the line length must not exceed LINE_LENGTH */ /* Return 0 if printer not ready to print, else 1 */ *****/ </pre>
Identifiers	Use descriptive names for all variable, function names, constants, and other identifiers. Avoid abbreviations or single-letter variables.
Identifier Example	<pre> Int number_of_students; /* This is GOOD */ Float: x4, j, ftave; /* This is BAD */ </pre>
Comments	- Document the code so the reader can understand its operation. - Comments should explain both the purpose and behavior of the code. - Comment variable declarations to indicate their purpose.
Good Comment	<pre>If(record_count > limit) /* have all records been processed? */</pre>
Bad Comment	<pre>If(record_count > limit) /* check if record count exceeds limit */</pre>
Major Sections	Precede major program sections by a block comment that describes the processing done in the next section.
Example	<pre> /***** /* The program section examines the contents of the array 'grades' and calcu- */ /* lates the average class grade. */ *****/ </pre>
Blank Spaces	- Write programs with sufficient spacing so they do not appear crowded. - Separate every program construct with at least one space.
Indenting	- Indent each brace level from the preceding level. - Open and close braces should be on lines by themselves and aligned.
Indenting Example	<pre> while (miss_distance > threshold) { success_code = move_robot (target_location); if (success_code == MOVE_FAILED) { printf("The robot move has failed.\n"); } } </pre>
Capitalization	- Capitalize all defines. - Lowercase all other identifiers and reserved words. - To make them readable, user messages may use mixed case.
Capitalization Examples	<pre> #define DEFAULT-NUMBER-OF-STUDENTS 15 int class-size = DEFAULT-NUMBER-OF-STUDENTS; </pre>

Evaluation criteria and suggestions

Evaluation criteria

Your standard must be

- complete
 - legible
-

Suggestions

Keep your standards simple and short.

Do not hesitate to copy or build on the PSP materials.

Coding Standard Template

Purpose	Guiar la implementación de programas en Java para el cálculo de regresión lineal y manejo de archivos.
Program Headers	Comenzar todos los archivos con un bloque de comentarios Javadoc descriptivo.
Header Format	<pre>/** * Asignación de Programa: PSP 3A * Nombre: [Emma Hernández Mendoza] * Fecha: [2025-11-23] * Descripción: [Descripción de la clase] */</pre>
Listing Contents	Provide a summary of the listing contents.
Contents Example	
Reuse Instructions	Utilizar comentarios Javadoc antes de cada método para explicar su función, parámetros (@param) y retorno (@return).
Reuse Example	<pre>/** * Calcula la suma de los productos X*Y * @param datosX array con los valores de X * @param datosY array con los valores de Y */ public void sumXY(double[] datosX, double[] datosY) { ... }</pre>
Identifiers	Use descriptive names for all variables, function names, constants, and other identifiers. Avoid abbreviations or single letter variables. - Clases: PascalCase (Mayúscula inicial). - Métodos: camelCase (minúscula inicial). - Variables: camelCase. En la clase matemática (EstimacionCorLineal), se utiliza notación húngara para indicar el tipo de dato (dbl para double, int para entero).
Identifier Example	- Clases: Logic3a, EstimacionCorLineal. - Variables (Estándar): fileName, fileContent, linesArray. - Variables (Matemáticas/Húngara): dblSumX, intN, dblRXY, dblB1.

(continued)

Coding Standard Template (continued)

Comments	• Usar comentarios de una línea (//) para explicar la lógica paso a paso dentro de los métodos. Usar Javadoc (/** */) para clases y métodos.
Good Comment	<pre>// Calcular numerador: (sumXY) - (n * promedioX * promedioY) double numerador = this.dblSumXY - (this.intN * this.dblAvgX * this.dblAvgY);</pre>
Bad Comment	<pre>// Cálculo double numerador = this.dblSumXY - (this.intN * this.dblAvgX * this.dblAvgY);</pre>
Major Sections	Separar los campos (atributos) de los métodos de acceso y otros métodos mediante comentarios de bloque simples.
Example	<pre>// // Métodos de acceso //</pre>
Blank Spaces	• Dejar líneas en blanco entre la declaración de variables y la lógica, y entre bloques lógicos distintos (como antes de un if o for).
Indenting	• Usar indentación de 4 espacios. Las llaves de apertura ({} van en la misma línea de la declaración (estilo K&R/Egyptian).
Indenting Example	<pre>for (int i = 0; i < linesArray.length; i++) { String line = linesArray[i].trim(); // Eliminar espacios en blanco // Saltar líneas vacías if (line.isEmpty()) { continue; } // ... }</pre>
Capitalization	• Capitalized all defines. • Lowercase all other identifiers and reserved words. • Messages being output to the user can be mixed-case so as to make a clean user presentation.
Capitalization Example	• Clases: App, Data, Input. • Métodos: readData, saveData, logic3a. • Constantes/Variables finales: final String result. ○ Capitalization Example EstimacionCorLineal (Clase), getDbSumXX (Método).